

Applied Big Data Engineering — Mini Project

Smart Grid Energy Monitoring & Billing

A Lambda-architecture data pipeline

Use Case 3

Group members

EG/2021/4598 — Karunarathna C.K

EG/2021/4520 — Gangadara M.G.K

EG/2021/4478 — Dharmathilaka M.D.S.K

25 September 2026

Contents

1. Introduction and use case	3
1.1 Requirements and how we meet them	3
1.2 Simulated clock	4
2. Architecture decision: Lambda, not Kappa	5
2.1 Why we rejected Kappa	5
2.2 The price we pay (trade-offs)	5
3. System architecture	6
4. Technology stack and justification	7
5. Data sources and ingestion	8
5.1 Streaming source: smart meters	8
5.2 Daily-batch source: tariff file	8
5.3 Ingestion	8
6. Processing	9
6.1 Speed layer: Spark Structured Streaming	9
6.2 Batch layer: Apache Airflow	9
6.3 Why the batch layer joins usage with tariffs	10
7. Storage and serving	11
7.1 Storage	11
7.2 Serving	11
8. Observability design	12
8.1 Alert rules	12
9. Results	13
9.1 Live dashboard	13
9.2 Daily billing report	14
9.3 Pipeline health	15
9.4 Failure and recovery tests	15
9.5 Automated tests	15
10. Limitations, trade-offs and production changes	16
10.1 Assumptions and simplifications	16
11. Individual contributions	17
Appendix A — Detailed architecture	18
Appendix B — How to run the project	18

1. Introduction and use case

This project builds a small but complete data platform for **Use Case 3: Smart Grid Energy Monitoring & Billing**. A power company wants to see, in real time, how much electricity each area of its grid is using and how much of it comes from solar panels. Once a day, it also wants to know what each household owes, using the electricity prices (tariffs) published that day.

The business question from the assignment is:

“What is the current grid load and renewable contribution by zone, and what will each household's bill look like once daily tariff data is applied to their consumption?”

The question has two halves, and they need very different things:

- **“Right now”** (grid load and renewable %) needs an answer within seconds. It can be slightly approximate.
- **“Each household's bill”** must be exact and repeatable, because it is money. It can wait until the day is over.

This difference drives the whole design (see section 2).

1.1 Requirements and how we meet them

Requirement (from the brief)	How the project meets it
Streaming source (Python), events every few seconds	meter_stream.py: 30 smart meters, one reading each every ~3 s
Daily-batch source (Python), one file per simulated day	tariff_batch.py: one tariff CSV per simulated day
Kafka ingestion (producers, topics, partitions)	Topic meter-readings, 3 partitions, key = meter_id
Stream processing (Spark Structured Streaming)	Cleaning, de-duplication, 1-hour windows, low-renewable alert
Orchestration (Airflow)	Daily billing DAG with 6 tasks
Queryable storage	PostgreSQL, plus a raw copy of all messages as files
Meaningful transformations (cleaning, joins, aggregation, windows)	All four: validation, dedup, windowed aggregation, join of usage with tariffs
API with live grid load / renewable mix by zone	FastAPI: GET /api/grid/current
Alert when renewable contribution is low	Spark LOW_RENEWABLE alert + Grafana alert rule
Daily billing and solar report per household	daily_bills table + CSV report + Grafana table + /api/bills
Structured logging and at least one alert / health rule	JSON logs in every service, /health, 4 Grafana alert rules
README, Docker Compose, tests	README.md, run_book.md, 9 containers, 38 automated tests

1.2 Simulated clock

Nobody can wait a real day for a bill, so time is sped up. All processing uses the time written inside each reading (event time), not the computer's clock.

Setting	Value
1 simulated day	5 real minutes (288 times faster)
1 meter reading	every 15 simulated minutes = about every 3 real seconds
First simulated day	2026-01-01
Random seed	42 — every run produces exactly the same data

2. Architecture decision: Lambda, not Kappa

We chose a **Lambda architecture**: the same data stream feeds two independent layers, each built for one half of the business question.

	“What is happening now?”	“What does each home owe?”
Needs	an answer within seconds	an exact, repeatable number
Can it be slightly off?	yes	no — it is money
Handled by	Speed layer: Spark	Batch layer: Airflow
Groups data by	area + 1 hour	household + 1 day

Both layers subscribe to the same Kafka topic independently. Spark reads it continuously; Airflow reads it in batches every minute and keeps its own **raw, unchangeable copy** of every message (the master dataset). The batch layer never uses anything Spark produces, so it can always recalculate any day from the original data.

2.1 Why we rejected Kappa

In a Kappa architecture everything is one stream: tariffs would also be published to Kafka, bills would come from a streaming job that joins two streams, and recalculating a day would mean replaying Kafka. We rejected it for three reasons:

1. **Bills must be exact and repeatable.** In a stream, results keep changing as late or duplicate readings arrive. A bill should be calculated once, after the day is complete, from all of its data.
2. **Recalculating is expensive.** Kappa recalculates by replaying Kafka, so Kafka would have to keep all history forever. We keep the history in the raw copy, and Kafka is only a short buffer (7 days by default).
3. **The tariff arrives as a daily file, not a stream.** A scheduled daily job fits it naturally; turning it into a stream adds work with no benefit.

2.2 The price we pay (trade-offs)

- **Two code paths.** The cleaning rules exist twice: in Spark, and in Python for the batch layer. An automated test checks that both reject exactly the same records.
- **Live numbers and bills can differ slightly.** For example, a re-sent (duplicate) reading is counted in the live hourly total but not in the bill.
- **More moving parts** than a single streaming job.

This is the normal Lambda trade-off: fast first, exact later.

3. System architecture

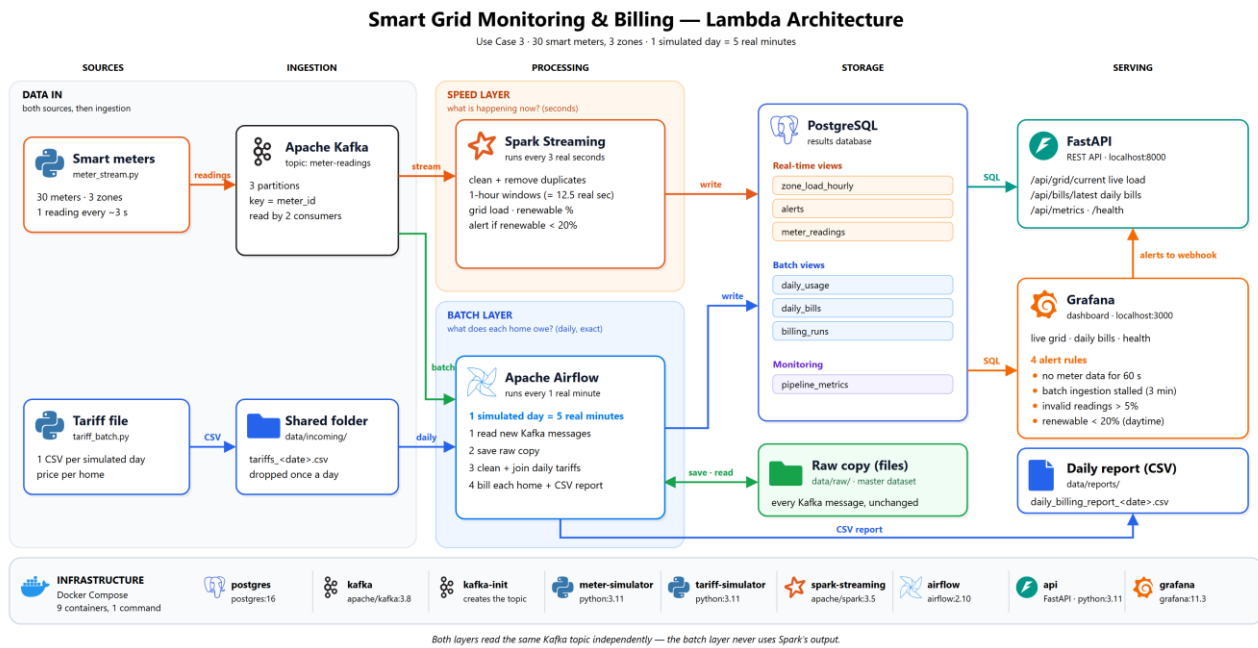


Figure 1 — Overall architecture (a more detailed version is in Appendix A)

Read the diagram from left to right:

1. **Sources.** The smart-meter simulator sends readings; the tariff simulator drops one CSV file per day.
2. **Ingestion.** Meter readings go to Kafka. The tariff file lands in a shared folder (data/incoming/).
3. **Processing.** Spark (speed layer, orange) reads Kafka every 3 seconds. Airflow (batch layer, blue) reads the same topic every minute, saves a raw copy, joins it with the tariff file and calculates the bills.
4. **Storage.** Results go to PostgreSQL. The raw copy of every Kafka message is kept as files (data/raw/).
5. **Serving.** FastAPI and Grafana read PostgreSQL. Airflow also writes a daily CSV report.
6. **Infrastructure.** All 9 services run as Docker containers, started with one command.

4. Technology stack and justification

Each tool was chosen for a concrete need of this use case, not for popularity.

Layer	Tool	Why it fits this project
Ingestion	Apache Kafka	Meters produce a never-ending stream. Kafka stores it safely (nothing is lost if a consumer stops) and lets two consumers read it independently. 3 partitions keyed by meter_id keep each meter's readings in order.
Stream processing	Spark Structured Streaming	Built-in event-time windows and watermarks, which we need because each reading carries its own timestamp and can arrive late. 3-second micro-batches are fast enough for live monitoring; checkpoints allow restarts.
Batch processing / orchestration	Apache Airflow	The batch layer is a scheduled job with ordered and parallel steps, retries and a visible run history — exactly what Airflow provides.
Storage (results)	PostgreSQL	Small, structured results that need SQL joins (usage + tariffs). Primary keys make re-runs safe (no duplicates). NUMERIC stores money exactly. For 30 meters, Cassandra or HDFS would add complexity without benefit.
Storage (master dataset)	Files (JSON lines)	An append-only, unchangeable copy of every Kafka message. In production this would be Parquet on S3 or HDFS.
Serving API	FastAPI	Small and simple, with automatic interactive documentation at /docs.
Dashboard + alerting	Grafana	Builds dashboards straight from PostgreSQL with no front-end code, and runs the alert rules. Everything is provisioned from files.
Packaging	Docker Compose	One command starts all 9 services with the same versions on any computer.

5. Data sources and ingestion

5.1 Streaming source: smart meters

meter_stream.py simulates **30 households in 3 areas** (north, central, south). 30%, 40% and 50% of the homes in those areas have solar panels. Every 15 simulated minutes each meter sends one message with exactly the fields from the brief:

```
{"meter_id": "M001", "household_id": "H001", "grid_zone": "north",
  "power_consumption_kwh": 0.21, "solar_generation_kwh": 0.64,
  "timestamp": "2026-01-01T10:15:00Z"}
```

To make the data realistic and to test the pipeline, the simulator also:

- follows a daily pattern: homes use more electricity in the morning and evening; solar only produces between 06:00 and 18:00;
- creates a **storm over one area from 10:00 to 14:00 every day** (north on day 1, central on day 2, south on day 3). Solar output collapses, so the low-renewable alert fires;
- sends about **2% bad records** (negative values, missing fields, an unknown area, broken JSON) and about **1% duplicates**;
- adds two Kafka **headers** to every message: trace_id (for tracing) and produced_at (to measure delay).

5.2 Daily-batch source: tariff file

tariff_batch.py plays the power company's billing system. At the start of each simulated day it drops one file, data/incoming/tariffs_<date>.csv, with one row per household:

```
household_id,tariff_rate,billing_tier,subsidy_flag
H001,0.1547,A,true
```

Column	Meaning
tariff_rate	price per kWh for that day (changes slightly every day)
billing_tier	A / B / C — higher-use homes pay a higher price and a bigger daily service charge
subsidy_flag	whether the home gets a 20% government discount on its energy charge

Every third day, one row is deliberately broken (a blank or negative price), to show that bad batch data is caught.

5.3 Ingestion

- **Kafka topic meter-readings**: 3 partitions, messages keyed by meter_id. Two independent consumers read it: Spark and Airflow (consumer group airflow-batch).
- **Shared folder data/incoming/**: the tariff file does not go through Kafka. It is a small daily file read by one job, so a message queue is not needed — exactly as the brief describes (“drops a new file once per simulated day”).

6. Processing

6.1 Speed layer: Spark Structured Streaming

Spark runs continuously and processes a **micro-batch every 3 seconds** (about 30 new readings). It runs two jobs side by side.

Job 1 — clean and store. Each message is parsed and checked. Bad records are saved to `rejected_readings` together with the reason. Duplicates are removed inside each batch (`dropDuplicates`) and across batches (the table's primary key is `meter_id` + `timestamp`). Clean readings are saved to `meter_readings`.

Check	Example of a bad record	Reason stored
valid JSON, meter and time present	<code>{"meter_id": "M0 (cut off)"}</code>	<code>malformed_or_missing_fields</code>
household present	<code>household_id</code> is null	<code>missing_household_id</code>
known area	<code>grid_zone = zone-x</code>	<code>unknown_grid_zone</code>
no negative values	<code>power_consumption_kwh = -0.2</code>	<code>invalid_consumption</code>

Job 2 — live totals per area. Clean readings are grouped into **1-hour tumbling windows per area** using event time. For each window Spark calculates the total consumption, total solar, net grid load (consumption minus solar) and **renewable %** (share of consumption covered by solar). A **1-hour watermark** means readings up to 1 simulated hour late are still counted. Results are upserted into `zone_load_hourly`, so the dashboard updates live.

Alert rule: between 09:00 and 15:00, once at least 75% of an hour's readings have arrived, if renewable % is below **20%**, Spark saves a `LOW_RENEWABLE` alert.

Example (real output): at 11:00 on 2026-01-01 (simulated), the storm area north had a renewable share of only 3.7%, so Spark raised a `LOW_RENEWABLE` alert. In the same hour the sunny south area used 6.7 kWh and its panels produced 11.9 kWh, so it was fully covered by solar (renewable 100%) and even sent 5.2 kWh back to the grid.

6.2 Batch layer: Apache Airflow

The Airflow DAG `smart_grid_daily_billing` runs **every real minute** (about 5 times per simulated day). It has 6 tasks; two of them run in parallel:

```
ingest_from_kafka → pick_day → [ process_raw_readings + load_tariffs ] → compute_bills
→ export_report
                        (these two run in parallel)
```

1. **ingest_from_kafka** (every run): notes where the topic ends right now, reads everything since the last run up to that point, saves each message unchanged to the raw copy (`data/raw/date=<day>/`), and only then commits the offsets. If a run fails, Kafka keeps the messages and the next run reads them again.
2. **pick_day**: finds the first simulated day that is finished (plus 1 hour for late readings), has a tariff file, and has not been billed. If there is none, the billing tasks are skipped for this run.
3. **process_raw_readings**: reads that day's raw copy and cleans it itself, with the same rules as Spark: rejects bad records, removes duplicates, and totals each home's day into `daily_usage`.
4. **load_tariffs** (in parallel with 3): checks each row of the tariff CSV; bad rows are rejected and logged.

5. **compute_bills**: SQL joins **daily_usage** with that **day's tariffs** on household and date, then applies the billing rules.
6. **export_report**: writes data/reports/daily_billing_report_<date>.csv and marks the day as done.

6.3 Why the batch layer joins usage with tariffs

The meter readings only say **how much** electricity a home used. The tariff file says **what it costs** that day. A bill needs both, so the two sources are joined on household_id and date — the “join between the two sources” required by the brief.

```
energy charge = electricity bought from the grid × price
solar credit  = solar electricity sent back to the grid × price × 50%
service charge = fixed daily charge by tier (A 0.50 / B 0.75 / C 1.00)
subsidy       = 20% of the energy charge, if eligible
total bill    = energy charge - solar credit + service charge - subsidy
```

Worked example (real output, H001 on 2026-01-01): bought 8.65 kWh from the grid and exported 2.19 kWh; rate 0.1547, tier A, subsidy yes. Energy 1.34 – solar credit 0.17 + service 0.50 – subsidy 0.27 = **1.40**.

If a home has no valid tariff row (for example because its row was broken), the join finds no match: the home is not billed with a wrong price, and it is reported as **unbilled**.

7. Storage and serving

7.1 Storage

Table / store	Filled by	What it holds
meter_readings	Spark	clean readings seen by the speed layer
rejected_readings	Spark	bad readings and why they were rejected
zone_load_hourly	Spark	live hourly totals per area (real-time view)
alerts	Spark	low-renewable alerts
data/raw/ (files)	Airflow	every Kafka message, unchanged — the master dataset
tariffs	Airflow	validated daily prices
daily_usage	Airflow	each home's cleaned daily totals
daily_bills	Airflow	one bill per home per day (batch view)
billing_runs	Airflow	one summary row per billed day
pipeline_metrics	Spark, Airflow	row counts, timings and delays (monitoring)

7.2 Serving

Endpoint (FastAPI, localhost:8000)	Returns
GET /api/grid/current	live grid load, solar and renewable % per area
GET /api/alerts	latest low-renewable alerts
GET /api/bills/latest, /api/bills/{date}	daily billing report (summary + bill per home)
GET /api/metrics	pipeline numbers: rows, rejected, duplicates, delay
GET /api/trace/{trace_id}	the full journey of one reading
GET /health	ok, or stale if no reading arrived for 60 s

Grafana (localhost:3000) shows one dashboard with three sections: the live grid, the daily billing report, and pipeline health. It also runs the alert rules (section 8).

8. Observability design

The pipeline is designed to be observable, not just functional: every stage reports what it did, and problems raise alerts automatically.

What	How	Why
Structured logs	Every service writes one JSON line per event, e.g. <code>{"service": "spark-streaming", "event": "readings_stored", "rows_in": 30, "rejected": 1}</code>	Easy to search; shows which stage and which batch a problem came from
Metrics	pipeline_metrics: rows in, valid, invalid, duplicate, processing time and end-to-end delay for every batch (Spark and Airflow). Exported by /api/metrics and shown in Grafana	Throughput, data quality and delay over time
Health check	/health returns ok, or stale if no reading arrived in 60 s	Shows at a glance whether data is flowing
Alert rules	4 Grafana rules (below); notifications are sent to the API webhook and logged	Problems are detected without anyone watching the screen
Tracing	Every reading has a trace_id in its Kafka header, stored in the database and written in the logs; /api/trace/<id> shows its journey	Follow one record: was it sent, rejected (why), which window and bill it went into

8.1 Alert rules

Rule	Fires when	Detects
No meter data received for 60 seconds	nothing reached the database for 60 s	live path failure (simulator, Kafka or Spark)
Batch ingestion has not run for 3 minutes	Airflow stopped reading Kafka	batch path failure
Invalid reading rate above 5%	too many bad records in 5 minutes	bad upstream data
Low renewable contribution in a grid zone	a LOW_RENEWABLE alert was raised	the business alert from the brief

9. Results

All numbers below come from a fresh run of the system (one fresh start; the tables cover the first 6 simulated days and the screenshots were taken later in the same run). Because the simulator is seeded, every run produces the same results.

9.1 Live dashboard

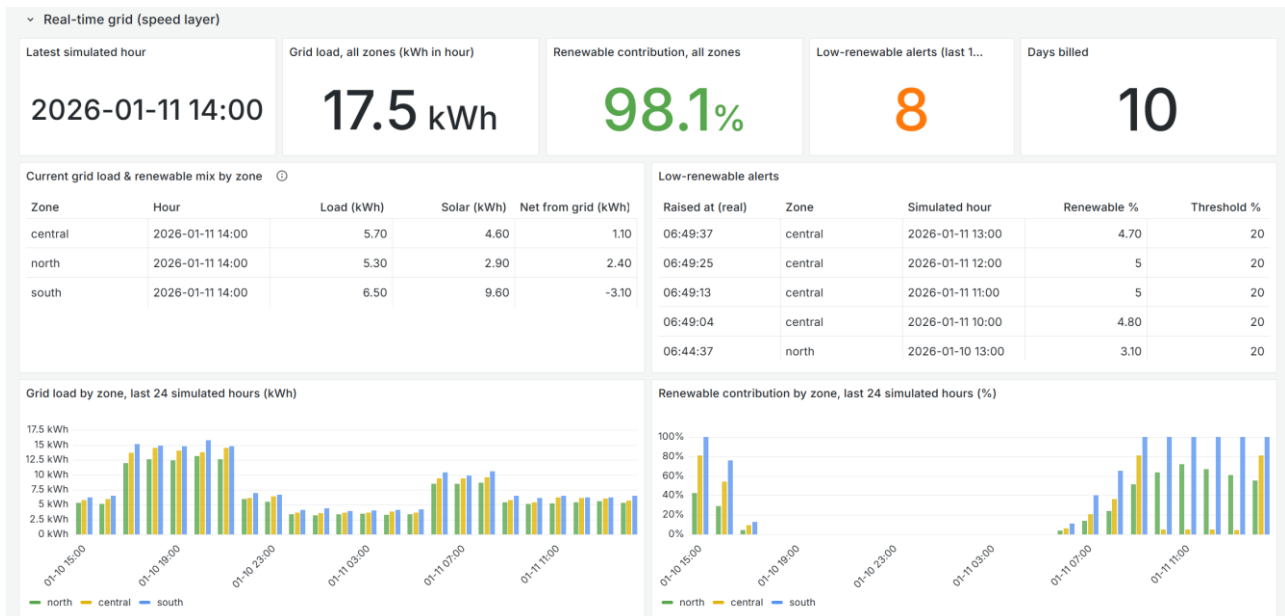


Figure 2 — Grafana, live grid: load and renewable % per area, alerts from the central-area storm (renewable about 5%), and 24-hour charts where central drops to almost zero at midday

Area	Hour (simulated)	Load (kWh)	Solar (kWh)	Renewable %
central	2026-01-07 06:00	9.51	0.6	6.3
north	2026-01-07 06:00	8.46	0.43	5
south	2026-01-07 06:00	10.01	1.06	10.6

Table — live output of GET /api/grid/current

Low-renewable alerts raised: north: 8 (renewable 3.0–3.8%); central: 8 (renewable 4.9–6.0%); south: 8 (renewable 7.3–9.1%). Each storm produces exactly 4 alerts (one per hour from 10:00 to 14:00) in the affected area, and the storm moves to a different area each day, so after 6 days each area has 8.

9.2 Daily billing report

▼ Daily billing report (batch layer)

Report date

2026-01-10

Households billed

30

Total billed

\$102.28

Unbilled households (missin...

0

Raw readings rejected by batch I...

34

Per-household bill & solar contribution (latest billed day)

Household	Zone	Used (kWh)	Solar (kWh)	From grid (kWh)	To grid (kWh)	Solar share %	Rate / kWh	Tier	Subsidy
H001	north	11.4	4.55	8.51	1.70	25.1	0.163	A	yes
H002	central	13.6	15.1	8.25	9.73	39.3	0.163	A	no
H003	south	14.8	17.5	8.94	11.7	39.5	0.163	A	no
H004	north	16.4	4.80	12.7	1.04	22.9	0.193	B	no
H005	central	18.6	15.2	11.5	8.04	38.4	0.193	B	no
H006	south	19.9	17.6	12.2	9.87	38.7	0.193	B	yes
H007	north	21.0	4.74	16.7	0.440	20.5	0.234	C	no
H008	central	23.7	15.0	15.3	6.64	35.4	0.234	C	no
H009	south	25.0	18.3	15.6	8.83	37.8	0.234	C	no
H010	north	12.0	0	12.0	0	0	0.163	A	no

Figure 3 — Grafana, daily billing report for the latest billed day: 30 homes billed, bill per home

Day	Homes billed	Unbilled	Tariff rows rejected	Raw records	Rejected readings	Duplicates	Total billed
2026-01-01	30	0	0	2,896	41	37	98.62
2026-01-02	30	0	0	2,901	40	30	102.41
2026-01-03	29	1	1	2,898	39	31	96.34
2026-01-04	30	0	0	2,900	38	32	101.04
2026-01-05	30	0	0	2,903	40	38	102.79
2026-01-06	29	1	1	2,891	37	29	101.82

Table — billing_runs: one row per billed day

Days 2026-01-03 and 2026-01-06 (every third day) show the batch validation working: one tariff row had a broken price, it was rejected, and that household was reported as unbilled instead of being billed with a wrong price. The raw records always add up: raw records = valid readings + rejected + duplicates.

Home	Area	Used (kWh)	From grid	To grid	Rate	Tier	Subsidy	Total bill
H001	north	11.84	8.65	2.19	0.1547	A	yes	1.40
H002	central	13.12	8.04	6.39	0.1547	A	no	1.25
H003	south	15.04	8.95	12.48	0.1547	A	no	0.91
H004	north	16.34	12.40	1.45	0.1837	B	no	2.90
H005	central	17.82	11.35	5.17	0.1837	B	no	2.36
H006	south	20.19	12.47	10.30	0.1837	B	yes	1.63

Table — first rows of daily_billing_report_2026-01-01.csv

9.3 Pipeline health

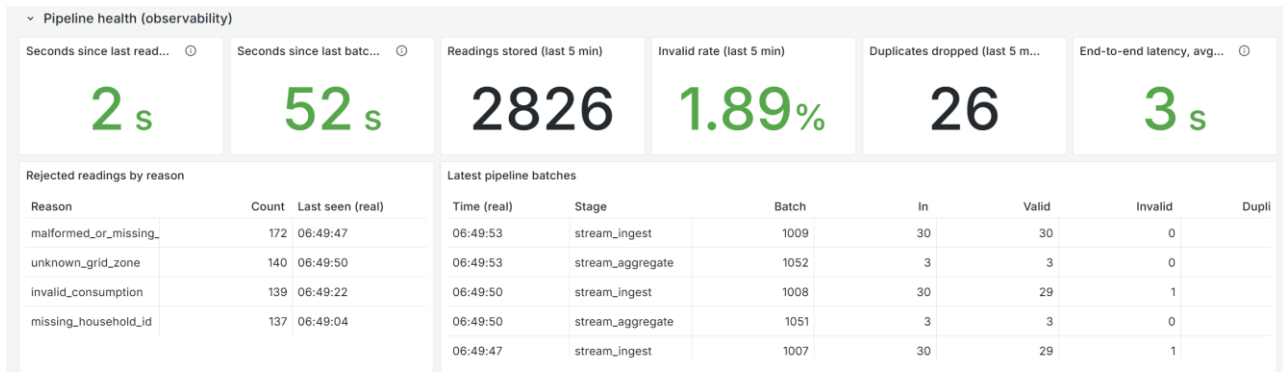


Figure 4 — Grafana: pipeline health (freshness, invalid rate, delay, rejected readings)

Measure	Result
Readings stored by Spark	17,781
Readings rejected by Spark	341 (malformed_or_missing_fields 96, unknown_grid_zone 86, missing_household_id 85, invalid_consumption 74)
Invalid rate (last 5 min)	1.82% (injected: about 2%)
Spark micro-batch time (average)	1581 ms (trigger every 3 s)
End-to-end delay, meter to database (average)	3239 ms
Airflow ingest_from_kafka (average run)	570 messages in 1180 ms, every minute
Health check	ok

9.4 Failure and recovery tests

We deliberately stopped parts of the system to check that failures are detected and that no data is lost.

Test	What happened
Stop the meter simulator	“No meter data” alert fired after about 2 minutes and /health reported stale. After restarting, the missed readings were re-sent and the alert cleared within seconds.
Stop Spark (speed layer)	Live views stopped updating, but Airflow still billed the next day (30 homes, same total as always). The batch layer does not depend on Spark.
Stop Airflow for 2 minutes	Messages waited in Kafka. On restart, the first run read the 1,700 waiting messages in 1.4 seconds — nothing was lost.
Re-run the same day	Recomputed from the raw copy; bills identical (idempotent).

9.5 Automated tests

38 automated tests (pytest) check the data sources, the real Spark transformation code, the batch layer's cleaning and daily totals, and the bill formula. One test checks that **Spark and the batch layer reject exactly the same records**, which protects against the main risk of having two code paths.

10. Limitations, trade-offs and production changes

Limitation in this project	What we would do at production scale
Simulated data; prices, charges and subsidy are simple made-up rules	Real meter feeds and the company's real billing rules
30 meters, 1 Kafka broker, Spark on one machine	Kafka cluster (replication factor 3), Spark on a cluster
Raw copy is JSON lines on a local disk	Parquet files on S3/HDFS, partitioned by date; batch processing in Spark
Airflow reads Kafka itself; if stopped for more than 7 days, Kafka would delete messages	A dedicated ingestion tool (e.g. Kafka Connect) writing to storage
Airflow runs every minute because of the fast clock	Run once a day (e.g. at 01:00), waiting for the tariff file
Live totals are approximate (count duplicates; skip data more than 1 hour late)	Acceptable by design — the daily bill is exact
Messages without a readable timestamp cannot belong to any day	Send them to a dead-letter topic for manual review
Demo passwords in docker-compose.yml; no API login	Secret manager and authentication
Metrics in PostgreSQL; alerts to a webhook	Prometheus, Loki/ELK, OpenTelemetry tracing, alerts by email/Slack

10.1 Assumptions and simplifications

- Simulated time: 1 day = 5 real minutes; every timestamp is simulated event time.
- One storm per day in a rotating area, to demonstrate the alert.
- Bad records and duplicates are injected on purpose to exercise cleaning.
- Money is rounded to 2 decimals and stored as NUMERIC.

11. Individual contributions

The work was split into three parts, one per member. Each member built their part, wrote its tests, and wrote the matching sections of this report.

Member	Area	What they did
Karunarathna C.K (EG/2021/4598)	Data sources and infrastructure	Built the smart-meter simulator and the daily tariff file simulator, the simulated clock and household list, the Kafka topic setup, and the Docker Compose setup for all 9 services. Wrote the simulator tests and sections 1, 5 and Appendix B.
Gangadara M.G.K (EG/2021/4520)	Processing (speed and batch layers)	Built the Spark streaming job (cleaning, removing duplicates, hourly windows, alerts) and the Airflow billing job (reading Kafka, saving the raw copy, joining with tariffs, calculating bills). Wrote the processing and billing tests and sections 2, 4 and 6.
Dharmathilaka M.D.S.K (EG/2021/4478)	Storage, serving and observability	Designed the PostgreSQL tables, built the FastAPI service, the Grafana dashboard and alert rules, and the logging, metrics, health check and tracing. Made the architecture diagrams and documentation, and wrote sections 3, 7, 8, 9 and 10.

Appendix A — Detailed architecture

Smart Grid Energy Monitoring & Billing — Lambda Architecture

Use Case 3 - 30 smart meters in 3 zones - 1 simulated day = 5 real minutes - all 9 services run with docker compose

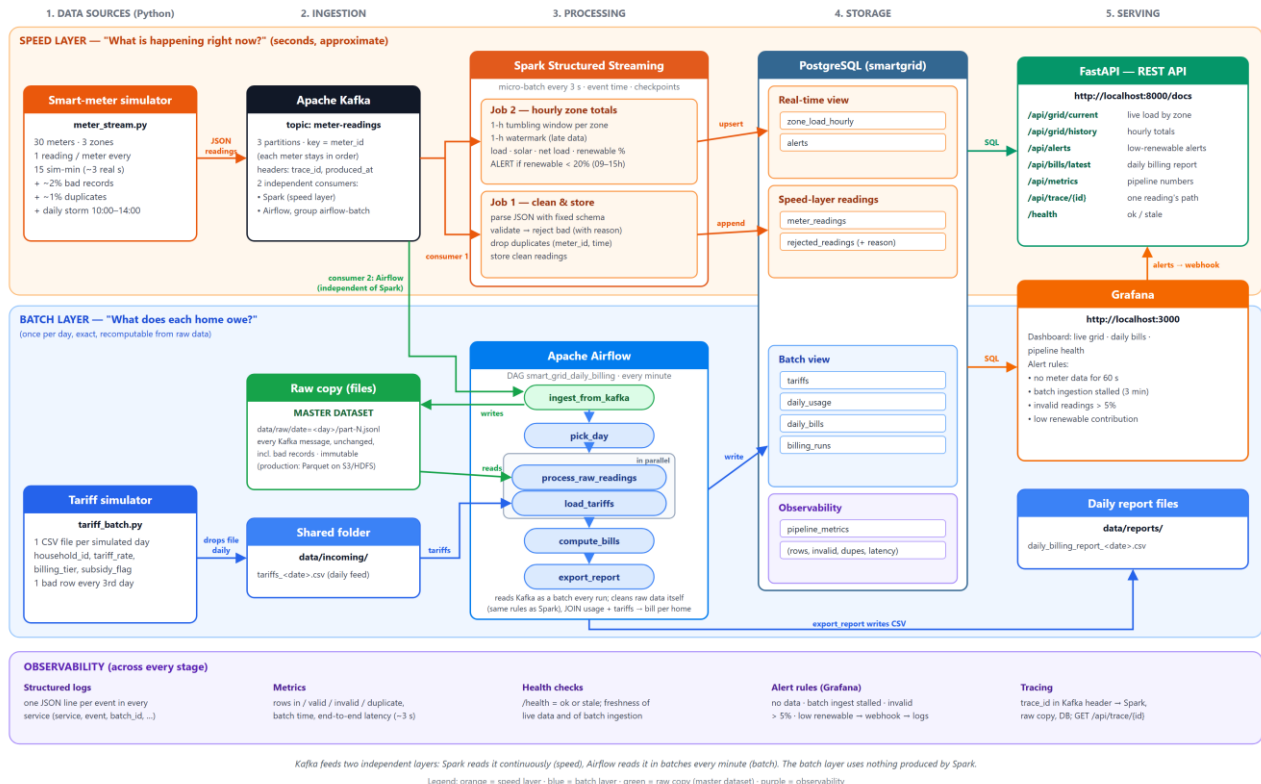


Figure A1 — Detailed architecture with every task, table and alert

Appendix B — How to run the project

Requirements: Docker Desktop and about 4 GB of free memory. Full step-by-step guide: `run_book.md` in the repository.

```
docker compose up -d --build      # start all 9 services
docker compose ps                 # check status
curl localhost:8000/health        # is data flowing?

docker compose down -v           # full reset (with: rm -rf data)
python -m pytest                  # run the 38 tests
```

UI	Address	Login
Grafana dashboard	<code>http://localhost:3000</code>	view without login; admin / admin to edit
API documentation	<code>http://localhost:8000/docs</code>	—
Airflow	<code>http://localhost:8080</code>	admin / admin